



Cours 02 – Notion 2 : Instances et Constructeurs

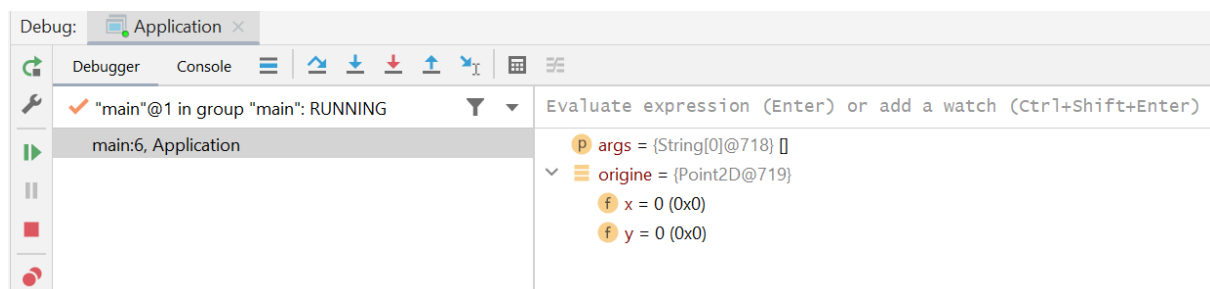
2.2.1 Définitions d'une instance

Comme nous l'avons défini lors du premier cours, des variables sont utilisées pour contenir des données. Afin de créer une nouvelle instance d'une classe, le constructeur de la classe doit être appelé en combinaison avec la comme `new`. Ce constructeur possède le nom de la classe. Si aucun constructeur n'est défini dans la classe elle-même, le compilateur de Java en définit un pour vous lors de la compilation de votre application.

```
// Définition d'un classe sans constructeur.  
public class Point2D  
{  
    public int x;  
    public int y;  
}
```

```
// utilisation d'un constructeur par défaut.  
public class Application  
{  
    public static void main(String[] args)  
    {  
        Point2D origine = new Point2D();  
    }  
}
```

Comme on le voit dans cet exemple, le "main" peut créer un objet de type Point2D, même si aucune fonction nommée Point2D n'a été définie dans la classe.



On remarque que dans cette situation, les attributs de la classe sont automatiquement initialisés à 0, ou l'équivalent. (Chaîne de caractère vide pour les String (""), false pour les booléens.)

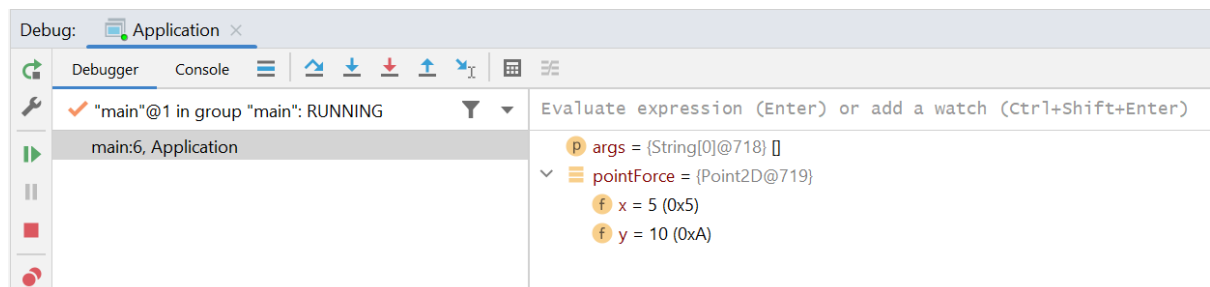
Cependant, il est beaucoup plus fréquent de définir soit même la manière dont l'on veut qu'un objet soit créé. Pour y arriver, la classe doit définir une méthode, qui possède le même nom que la classe. Il s'agit alors d'un constructeur spécialisé et des paramètres peuvent être envoyés au constructeur.

```
// Définition d'une classe avec constructeur spécialisée.
//
public class Point2D
{
    public int x;
    public int y;

    public Point2D(int xInitial, int yInitial)
    {
        this.x = xInitial;
        this.y = yInitial;
    }
}
```

```
// Utilisation du constructeur spécialisé.
//
public class Application
{
    public static void main(String[] args)
    {
        Point2D pointForce = new Point2D(5, 10);
    }
}
```

Comme on le voit dans cet exemple, le "main" peut créer un objet de type Point2D avec la méthode définie dans la classe.



On remarque que dans cette situation, les attributs du point créé possèdent les valeurs qui ont été transmises au constructeur.

2.2.2 Les constructeurs spécialisés

Un constructeur spécialisé est une méthode publique qui permet de créer une nouvelle instance d'une classe. Lorsqu'il n'y a qu'une manière d'appeler ce constructeur, celui-ci a comme mandat d'initialiser les attributs de la nouvelle instance en fonction des paramètres.

```
// Les deux points A{0,5} et B{3,1} servent à définir l'orientation
// d'une force appliquée sur la membrure Fab d'un treillis.
// L'amplitude de la force appliquée dans cette membrure n'est pas
// connue.
public class Application
{
    public static void main(String[] args)
    {
        Point2D pointA = new Point2D(0, 5);
        Point2D pointB = new Point2D(3, 1);
        Force force1 = new Force(pointA, pointB, "Fab");
    }
}
```

```
public class Force
{
    /* ... */
    public Force(Point2D point1, Point2D point2, String nomForce)
    {
        //On construit la force avec son nom reçu.
        this.nom = nomForce;

        //La force est positionnée sur le point1.
        this.positionX = point1.x;
        this.positionY = point1.y;

        //Cette force est une inconnue dans une équation.
        this.module = 0;
        this.coefficient = 0;
        this.estConnue = false;

        //On détermine l'angle de la force avec la géométrie de la
        //membrure
        double dx = point2.x - point1.x;
        double dy = point2.y - point1.y;
        this.orientation = Math.atan2(dy, dx);
    }
}
```

Tout d'abord, lors de l'appel, les paramètres effectifs (pointA, pointB, "Fab") sont transférés, selon l'ordre, vers les paramètres formels (point1, point2, nomForce), peu importe s'il partage le même nom ou pas.

Ensuite, certains attributs de l'instance peuvent être initialisés :

- en reprenant directement la valeur d'un paramètre (ex : nom et pointAction) ;
- en conservant les valeurs par défaut (ex : module, estConnue et coefficient) ;
- en étant calculé à partir des valeurs des paramètres (ex : orientation).

L'on remarque que pour accéder à une propriété d'une instance la syntaxe suivante doit être employée :

```
//Syntaxe pour modifier ou consulter un attribut
this.nomPropriete = valeurInscrite;
valeurLue = this.nomPropriete;
```

De plus, il n'y a pas de confusion entre la propriété d'une instance et un paramètre qui porterait le même nom, puisque le préfixe `this` vient clarifier l'accès aux attributs appartenant à l'objet créé. Donc, dans ce cas-ci, le préfixe `this` représente la nouvelle force qui est créée.

2.2.3 La surcharge de méthode

Ce principe fondamental en programmation orientée objet permet de définir plusieurs méthodes possédant le même nom, mais ayant une liste de paramètres différente les unes des autres. Toutes ces méthodes tentent d'accomplir la même tâche, mais d'une manière légèrement différente, puisqu'elle ne possède pas les mêmes paramètres d'entrées. On illustrera le principe de surcharge avec l'étude des constructeurs, mais n'importe quelle méthode (ou action) peut être surchargée en Java.

Nous avons déjà exprimé le fait qu'un constructeur par défaut pouvait être créé automatiquement par Java si aucun constructeur n'était fourni dans une classe, ou encore, un programmeur peut définir un constructeur spécialisé pour une classe. Nous allons donc différencier trois types de constructeur :

- Le constructeur par défaut : L'objectif de ce constructeur est d'initialiser les propriétés d'un objet avec des valeurs par défauts (pas nécessairement 0). Il est préférable de définir soi-même ce constructeur et ne pas laisser Java s'en charger.
 - Ce constructeur ne reçoit pas de paramètre.
- Le constructeur par initialisation : L'objectif de ce constructeur est d'initialiser les propriétés d'un objet avec des valeurs passées en paramètres. Chaque attribut est généralement initialisé par un paramètre reçu.
 - Ce constructeur reçoit plusieurs paramètres, souvent des données de types natives.
- Le constructeur par copie : L'objectif de ce constructeur est d'initialiser les propriétés d'un objet avec les valeurs d'un autre objet. Chaque attribut est généralement initialisé par le contenu d'un objet reçu en paramètre.
 - Ce constructeur reçoit un paramètre qui est une autre instance de la même classe.

```
public class Application
{
    public static void main(String[] args)
    {
        //Exemple d'appel pour un constructeur par défaut
        Force force1 = new Force();

        //Exemple d'appel pour un constructeur par initialisation
        Point2D pointA = new Point2D(0, 5);
        Point2D pointB = new Point2D(3, 1);

        Force force2 = new Force(pointA, pointB, "Fab");

        //Exemple d'appel pour un constructeur par initialisation
        Force force3 = new Force(force2);
    }
}
```

Définition des constructeurs à la prochaine page.

```

public class Force
{
    /* ... */

    /*
     * Définition d'un constructeur par Copie
     */
    public Force()
    {
        //Tous les attributs de la force sont initialisées à zéro.
        this.nom = "";

        this.positionX = 0;
        this.positionY = 0;

        this.module = 0;
        this.orientation = 0;

        this.coefficient = 0;
        this.estConnue = false;
    }

    /*
     * Définition d'un constructeur par initialisation
     */
    public Force(Point2D point1, Point2D point2, String nomForce)
    {
        //On construit la force avec son nom reçu.
        this.nom = nomForce;

        //La force est positionnée sur le point1.
        this.positionX = point1.x;
        this.positionY = point1.y;

        //Cette force est une inconnue dans une équation.
        this.module = 0;
        this.coefficient = 0;
        this.estConnue = false;

        //On détermine l'angle de la force avec la géométrie de la membrure
        double dx = point2.x - point1.x;
        double dy = point2.y - point1.y;
        this.orientation = Math.atan2(dy, dx);
    }

    /*
     * Définition d'un constructeur par Copie
     */
    public Force(Force autreForce)
    {
        //Tous les attributs de la force sont initialisées avec les données
        //de la force reçue.
        this.nom = autreForce.nom;

        this.positionX = autreForce.positionX;
        this.positionY = autreForce.positionY;

        this.module = autreForce.module;
        this.orientation = autreForce.orientation;

        this.coefficient = autreForce.coefficient;
        this.estConnue = autreForce.estConnue;
    }
}

```